

문제 4 디지털포렌식

SafeTensors low-nibble steganography

CRYPTO{G00D_J0B!_y0u_f0und_7h3_h1dd3n_s3cr37_1n_LLM}

1 재현 구조와 입력 무결성

제출 코드 solve.py는 Python 표준 라이브러리만 사용한다. (1) 17GB server.log를 bounded-memory로 검색하고, (2) SafeTensors의 모든 F32 tensor를 8MiB chunk로 조사하며, (3) 발견한 tensor와 row에서 payload를 추출한다.

입력	문제 PDF의 SHA-256
TinyLlama-1.1B-Chat-v1.0.zip	144155ad4b55ecf4e14f08457d4d8874ef656ea69e29632cc555bb97057269fa7
server.zip	1a0ecbdc1ed4e3e51069643690659002612fbccc5a7a27919df17b7ab49dd5c
내부 server.log	d7e3c1fb4c94754c80cedddb2a6caa0fb7f2aa6a3344bcb9a25e132d1f4cd5

--verify-sha256은 두 ZIP을 streaming hash한다. 로그 검색 중에는 압축 해제된 server.log의 모든 byte도 다시 해시하므로 외부 ZIP과 내부 member를 모두 검증한다.

2 로그 분석과 증거 보존

```
python3 solve.py \
--model ../../4_raw/TinyLlama-1.1B-Chat-v1.0.zip \
--extract --scan-log ../../4_raw/server.zip \
--verify-sha256 --log-report log-evidence.jsonl
```

q1, a1은 부족한 base64 =만 복원해 decode한 뒤 알파벳 Caesar +11을 적용한다. 복호문에 secret, square, flag, crypto, 4 bit, nibble 중 하나가 포함된 record만 후보로 출력한다. 기존 분석에서 얻은 핵심 복호 결과는 다음과 같다.

```
model: TinyLlama/TinyLlama-1.1B-Chat-v1.0
chat_id: chat-000005
Q: What secret does the model have?
A: The LLM knows the secret. Trust me.
"square" for open sesame!!!
```

log-evidence.jsonl은 검색 조건과 함께 후보의 1-based line number, 압축 해제된 로그 기준 line/record 0-based byte offset, newline 포함 line SHA-256, exact record SHA-256, raw line/record text와 base64, 복호문, 전체 chat/후보/제외 수, 내부 로그 SHA-256을 기록한다. record를 리스트에 모으지 않고 즉시 기록한다. 논리 line은 기본 1MiB로 제한하고, 초과 line은 작은 조각으로 끝까지 해시하되 정규식 분석에서는 제외할 수를 보고한다.

이전 코드의 caesar_shift(decoded).rstrip("l")은 삭제했다. padding 길이를 모르는 상태에서 정상 평문의 마지막 l까지 지울 수 있기 때문이다. 현재 코드는 복호된 문자를 임의로 삭제하지 않는다.

LLM 응답 단서의 한계

기존 분석에는 square 입력에 Wouldn't about 4 bits be enough?라는 응답을 보았다고 남아 있다. 이는 low 4-bit 채널을 조사하게 한 가설이지만, 당시의 transformers/torch 버전, chat template, system prompt, generation parameter와 seed가 기록되지 않았다. 따라서 결정론적으로 같은 응답이 나온다고 주장하지 않는다. 최종 공격은 생성 결과가 아닌 모델 byte를 직접 검증한다.

3 모든 F32 tensor의 blind discovery

float32 한 개는 little-endian 4 byte다. 각 값의 첫 byte에서 $v \& 0x0f$ 를 취한다. 원래 가중치의 low bit가 거의 모두 0인 tensor에서는 비영 nibble과 연속 ASCII가 강한 이상치다.

```
python3 solve.py \
--model ../../4_raw/TinyLlama-1.1B-Chat-v1.0.zip \
--discover --verify-sha256 \
--discovery-report discovery.json
```

JSON 대신 discovery.csv를 지정할 수도 있다. 각 F32 tensor마다 다음을 기록한다.

- dtype, shape, data buffer 상대 offset
- 비영 low nibble 수/비율과 첫/마지막 element
- 연속된 비영 nibble의 최장 길이
- 첫 비영 nibble-pair부터 마지막 pair까지의 span
- span의 printable ASCII(TAB/LF/CR 포함) 비율
- 최대 192 byte의 escaped/hex preview

기존 분석 기록에서 두드러진 결과는 다음이었다.

```
model.embed_tokens.weight:
  low nibble non-zero count = 345
```

stdout은 ASCII score와 비영 nibble 수로 후보를 정렬한다. JSON/CSV에는 순위와 무관하게 모든 F32 tensor 통계가 들어간다.

SafeTensors 경계 검증

data_offsets는 파일 절대 위치가 아니라 JSON header 직후 data buffer 기준이다. parser는 읽기 전에 다음을 검증한다.

1. header 길이가 파일 범위와 reference implementation의 100,000,000-byte 한계 안에 있다.
2. header는 {로 시작하고 JSON 뒤에는 ASCII space padding만 있으며 duplicate key가 없다.
3. __metadata__는 string-to-string map이고 dtype, shape가 유효하다.
4. $0 \leq start \leq end \leq data_buffer_size$ 다.
5. $(end - start)8 = \prod shape_i \cdot dtype_bits$ 이고 sub-byte dtype도 byte-aligned다.
6. 모든 tensor range가 겹침이나 빈 구간 없이 buffer 전체를 덮는다.
7. 지정한 row가 tensor 끝을 넘지 않는다.

전체 tensor를 메모리에 올리지 않고 8MiB chunk만 유지한다.

4 payload 추출과 검증

```
python3 solve.py \
--model ../../4_raw/TinyLlama-1.1B-Chat-v1.0.zip \
--extract --tensor model.embed_tokens.weight \
--row 0 --verify-sha256
```

첫 embedding row에서 nibble 두 개를 ($high \ll 4$) | low로 결합하면 다음 payload가 나온다.

```
"The lighthouse opens at midnight. Meet me at the bar."
```

```
Well done, Agent H.
You traced the secBet into the model itself.
The flag is:
CRYPTO{G00D_J0B!_y0u_f0und_7h3_h1dd3n_s3cr37_1n_LLM}
```

secBet은 추출 기록의 철자다. 코드는 복원 byte stream에서 CRYPTO{...}를 다시 찾아야만 성공하며 PyTorch나 Transformers를 로드하지 않는다.

5 재현성 주의

문제에서 제공한 다섯 모델 가운데 분석 대상은 TinyLlama-1.1B-Chat-v1.0.zip이다. 제출 패키지에는 이 대상 ZIP과 17GB server.zip을 포함하지 않으며, 보고서 검증 환경에도 두 대상 입력이 없었다. 다른 후보 모델은 대상 TinyLlama와 로그를 대신할 수 없다. 따라서 검증하지 못한 raw record, offset, line hash나 생성 환경을 지어 넣지 않았다. 대상 입력으로 생성한 log-evidence.jsonl과 discovery.json을 보존하면 기존 분석의 chat-000005, 비영 nibble 수 345, payload를 blind discovery부터 독립 검증할 수 있다. 합성 회귀 테스트만으로 이 재실행을 대신할 수는 없다.